3.1.2 DBT - Run DBT

Last updated by | Marta Pereira | Feb 26, 2026 at 11:45 PM GMT

Contents

- Run DBT
  - How to Run
  - Where to run
    - Local with Spark Container
    - Local using Databricks' Resources
    - Databricks Workflow
      - Possible Problems

Run DBT

How to Run

DBT commands:

- `dbt run` : Runs the models on the default target profile.
- `dbt run --select silver` : Runs the models on the 'silver' folder.
- `dbt run --target prod` : Runs DBT on the prod target.
- `dbt seed` : Runs the seeds.
- `dbt test` : Runs the tests.
- `dbt build` : Runs the models, seeds, and tests.
- `dbt build --exclude "/seeds"` : Runs the models and tests.

Note: With `dbt build` if a test fails for a certain model, any model that depends on it will get skipped.

Where to run

To run DBT for Databricks you have 4 options:

1. Run with local Spark container
2. Run locally using Databricks' resources
3. Run on a Databricks Workflow with a **dbt task**
4. Run with DBT Wrapper

Local with Spark Container

You can run a Spark container locally which will allow you to test your models quickly, though with a number of limitations when compared to Databricks:

- **No catalogs**: Local Spark has a single database with a number of schemas.
- **No Python models**: There is no Spark sessions and thrift (the connection method) only supports SQL.

Create the Container

To create this Container:

1. Go to **utils/Local Testing Environment** in the **dbt-template** repository .
2. Build the container: `docker build . -t localspark`
3. Run and connect to the container: `docker run -p 10000:10000 -it localspark bash`
4. Install the required DBT libraries: `python3 -m pip install dbt-core dbt-spark "dbt-spark[PyHive]"`

Connect to the Container

To be able to connect to the database with:

1. Run thrift server: `./spark/sbin/start-thriftserver.sh --driver-memory 4G --executor-memory 4G --executor-cores 2 --num-executors 1 --conf spark.driver.maxResultSize=2G`
2. Add a target to your **profiles.yml** like:

```yaml
local:
      type: spark
      method: thrift
      schema: default
      host: localhost
      port: 10000
      server_side_parameters:
        "spark.driver.memory": "4g"
```

To stop the thrift server:

1. Find the **Java** process ID: `ps -a`
2. Kill the process: `kill <process_id>`

Note: You can also connect to the Database using DBeaver (or other similar software) by creating a connection using the **Apache Spark** driver with **localhost** as the host and **10000** as the port. The credentials may be left blank.

Run code inside the Container

You can run python code inside the container, though it **cannot be run at the same time as the thrift server is activated**. To do so:

1. Launch pyspark: `/spark/bin/pyspark --driver-memory 3G --executor-memory 3G`
2. Run any code you want.
3. Close pyspark: `exit()`

Local using Databricks' Resources

You can run the DBT commands locally while still using Databricks' resources making it easier to test updates to the project:

1. Open **cmd** on the DBT project's root.
2. Install the **dbt-core** and **dbt-databricks** libraries with `pip install dbt-core dbt-databricks` .
3. Set the environmental variables for the **Client ID** and **Client Secret** for authentication by running `set CLIENT_ID=<client-id>` and `set CLIENT_SECRET=<client-id>` .
4. Test the connection with `dbt debug` . (If the All Purpose Compute Cluster isn't on, this step may take several minutes to run).

5. Run with `dbt run`.

Note: If you're having issues with dbt versions, it may make sense to create a python virtual environment for the dbt libraries and run the project using it. For information on how to create it, check the Install packages in a virtual environment using pip and venv page.

Databricks Workflow

Running the project on a Databricks Workflow is a more extensive endeavour than locally.

Connect Databricks Workspace to Git Repository

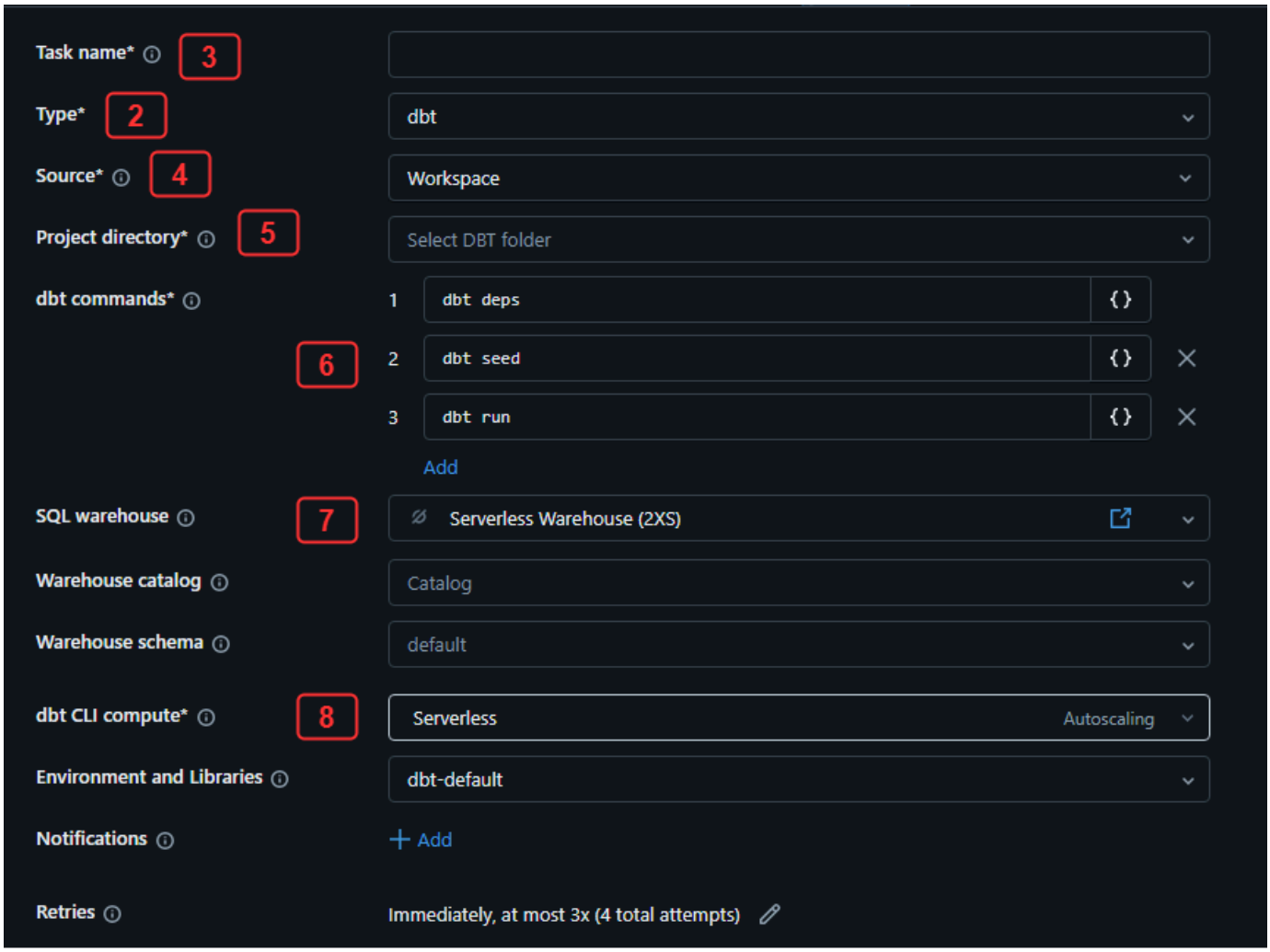
To easily access and alter the DBT project from within Databricks, you can link a folder in a Workspace to a specific branch of a Git Repository. To do so:

1. Go to the **Workspace** section.
2. Navigate the the folder you want to link the code to.
3. Click on **Create**.
4. Select **Git folder**.
5. Fill in the information.
6. Click on **Create Git folder**.
7. (Optional) If your Databricks user doesn't have permissions to the Git Repository, follow the instructions to use a different authentication like a Personal Access Token.

Create the DBT Task

To create a dbt task to run a DBT project, the following steps are required:

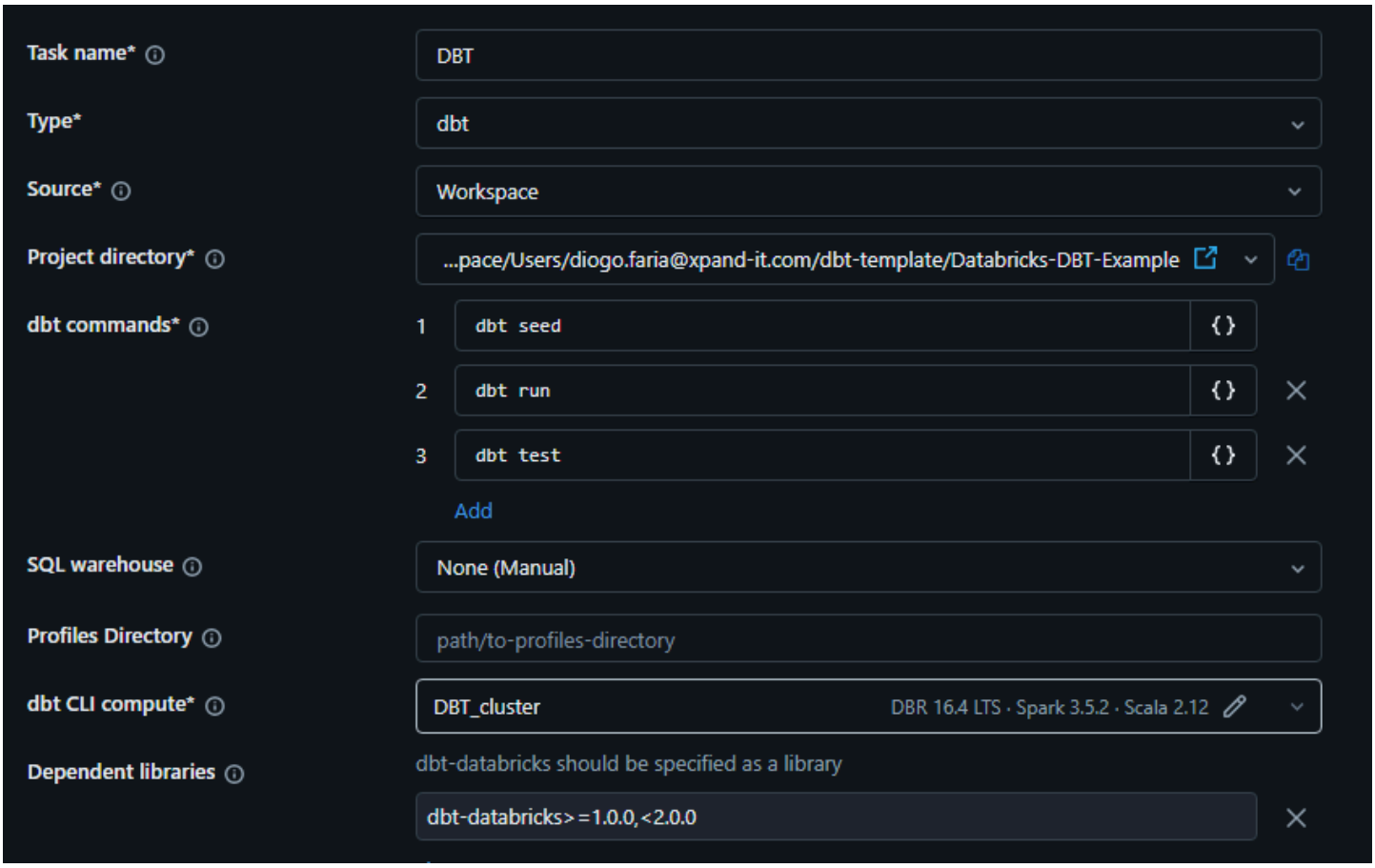
1. Create a **Workflow**.
2. Select the task type to be **dbt**.
3. Add the **task name**.
4. Add the **source type** (keep as **Workspace** to use the previously linked repository).
5. Select the **folder**.
6. Review the **dbt commands**.
7. Select the **SQL warehouse**. To define the SQL Warehouse or All Purpose Compute in the project's file, you need to select **None (Manual)** and have a **profiles.yml** file. If you do so and the **Warehouse catalog** and **Warehouse schema** options aren't removed as shown in the Example Task check the SQL Warehouse UI not appearing as it should section.
8. Select the **compute** or create a new job compute. **Always use the configured pool when creating a new job compute.**



9. Press on **Create task**.

Example Task

This example contains a functioning DBT task that points to a project in a DBT workspace that has been synced with an Azure DevOps Repository:



Note: If the **profiles.yml** is in the project's root, it isn't necessary to pass its path in the **Profiles Directory** field.

Pass Secrets to DBT

For authentication it may make sense to pass secrets to DBT to use on the **profiles.yml** if necessary. To do so, it's recommended to save the secrets to a Databricks Scope and, when running the workflow, pass the secret values as environmental variables:

1. Click on the **Edit** button for the Job Compute.
2. Scroll down and open the **Advanced** section.
3. Open the **Spark** tab on left-hand side.
4. In the **Environmental variables** text-box add the secrets:

```
CLIENT_ID={{secrets/<secret-scope>/<client-id-secret-name>}}
CLIENT_SECRET={{secrets/<secret-scope>/<client-secret-secret-name>}}
```

5. Access each secret in the **profiles.yml** targets for authentication:

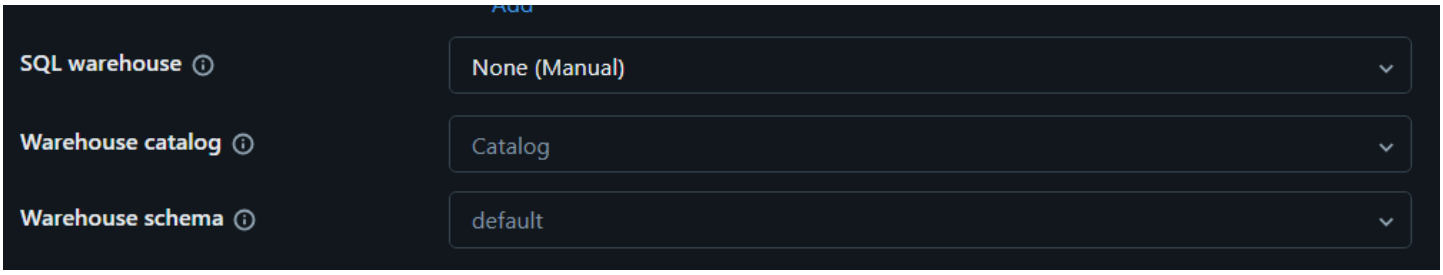
```
azure_client_id: "{{ env_var('CLIENT_ID') }}"
azure_client_secret: "{{ env_var('CLIENT_SECRET') }}"
```

Possible Problems

SQL Warehouse UI not appearing as it should

When you change the **SQL warehouse** to **None (Manual)** after it had an actual SQL warehouse selected, the UI may not revert to the correct state and still have the **Warehouse catalog** and **Warehouse schema**

fields, which will lead to an error when trying to save the pipeline: **No result returned**.



To solve this issue and save the task on the **None (Manual)** option:

1. Save the **dbt task** using a SQL warehouse (any warehouse - it doesn't matter as it won't be used).
2. Open the pipeline as a **YAML**.
3. Delete the **warehouse_id** property.
4. **Save**.

The UI should now be correct: